

Brute Force - Searching & String Matching

Sequential Search, Brute Force String Matching

Dr. SRIRAMYA

P.Bhanu PRAKASH (192425263)

Sequential Search

Question 1

Write a program to implement Sequential Search to find a given element in an array. Input:

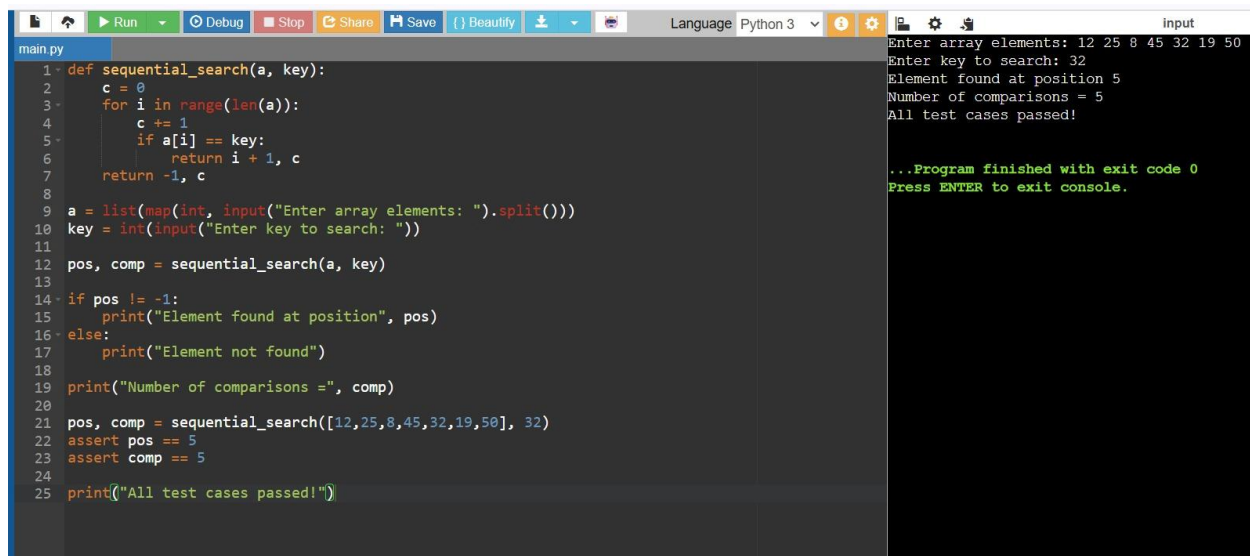
Array: 12, 25, 8, 45, 32, 19, 50

Key: 32

Output:

Element found at position 5

Number of comparisons = 5



```
main.py
1 def sequential_search(a, key):
2     c = 0
3     for i in range(len(a)):
4         c += 1
5         if a[i] == key:
6             return i + 1, c
7     return -1, c
8
9 a = list(map(int, input("Enter array elements: ").split()))
10 key = int(input("Enter key to search: "))
11
12 pos, comp = sequential_search(a, key)
13
14 if pos != -1:
15     print("Element found at position", pos)
16 else:
17     print("Element not found")
18
19 print("Number of comparisons =", comp)
20
21 pos, comp = sequential_search([12,25,8,45,32,19,50], 32)
22 assert pos == 5
23 assert comp == 5
24
25 print("All test cases passed!")
```

input

Enter array elements: 12 25 8 45 32 19 50
Enter key to search: 32
Element found at position 5
Number of comparisons = 5
All test cases passed!

...Program finished with exit code 0
Press ENTER to exit console.

Question 2

Implement Sequential Search for an unsuccessful search.

Input:

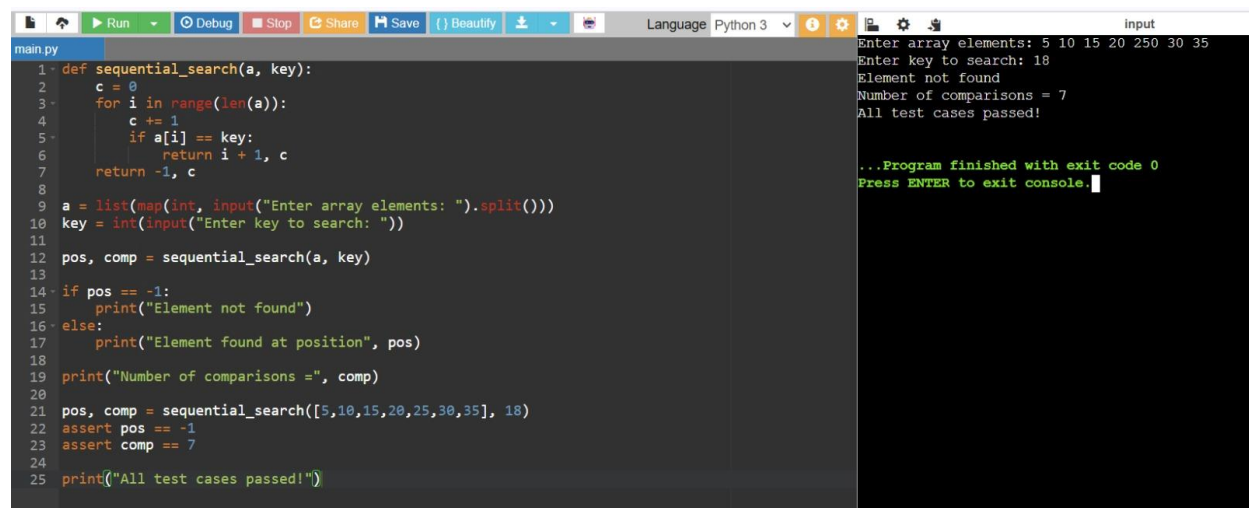
Array: 5, 10, 15, 20, 25, 30, 35

Key: 18

Output:

Element not found

Number of comparisons = 7



The screenshot shows a Python IDE with a file named 'main.py'. The code defines a 'sequential_search' function that takes an array 'a' and a 'key' as input. It initializes a counter 'c' to 0 and iterates through the array. If the key is found, it returns the index 'i' and the counter 'c'. If not found, it returns -1 and 'c'. The main part of the code prompts the user to enter array elements and a key to search for. It then calls the 'sequential_search' function with the input array [5, 10, 15, 20, 25, 30, 35] and key 18. The output shows that the element was not found and the number of comparisons was 7. The program finished with exit code 0.

```
1 def sequential_search(a, key):
2     c = 0
3     for i in range(len(a)):
4         c += 1
5         if a[i] == key:
6             return i + 1, c
7     return -1, c
8
9 a = list(map(int, input("Enter array elements: ").split()))
10 key = int(input("Enter key to search: "))
11
12 pos, comp = sequential_search(a, key)
13
14 if pos == -1:
15     print("Element not found")
16 else:
17     print("Element found at position", pos)
18
19 print("Number of comparisons =", comp)
20
21 pos, comp = sequential_search([5,10,15,20,25,30,35], 18)
22 assert pos == -1
23 assert comp == 7
24
25 print("All test cases passed!")
```

input
Enter array elements: 5 10 15 20 25 30 35
Enter key to search: 18
Element not found
Number of comparisons = 7
All test cases passed!
...Program finished with exit code 0
Press ENTER to exit console.

Question 3

Write a program to find the first occurrence of an element using Sequential Search.

Input:

Array: 10, 25, 15, 25, 30, 25, 40

Key: 25

Output:

First occurrence at position 2

```
main.py
1 def first_occurrence(a, key):
2     for i in range(len(a)):
3         if a[i] == key:
4             return i + 1
5     return -1
6
7 a = list(map(int, input("Enter array elements: ").split()))
8 key = int(input("Enter key to search: "))
9
10 pos = first_occurrence(a, key)
11
12 if pos == -1:
13     print("Element not found")
14 else:
15     print("First occurrence at position", pos)
16
17 pos = first_occurrence([10,25,15,25,30,25,40], 25)
18 assert pos == 2
19
20 print("All test cases passed!")
```

```
input
Enter array elements: 10 25 15 25 30 25 40
Enter key to search: 25
First occurrence at position 2
All test cases passed!

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 4

Implement Sequential Search to find all occurrences of a given element.

Input:

Array: 7, 12, 7, 25, 18, 7, 30, 7

Key: 7

Output:

Occurrences at positions:

1, 3, 6, 8

Total occurrences = 4

```
main.py
1 def first_occurrence(a, key):
2     for i in range(len(a)):
3         if a[i] == key:
4             return i + 1
5     return -1
6
7 a = list(map(int, input("Enter array elements: ").split()))
8 key = int(input("Enter key to search: "))
9
10 pos = first_occurrence(a, key)
11
12 if pos == -1:
13     print("Element not found")
14 else:
15     print("First occurrence at position", pos)
16
17 pos = first_occurrence([10,25,15,25,30,25,40], 25)
18 assert pos == 2
19
20 print("All test cases passed!")
```

```
input
Enter array elements: 10 25 15 25 30 25 40
Enter key to search: 25
First occurrence at position 2
All test cases passed!

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 5

Write a program to count:

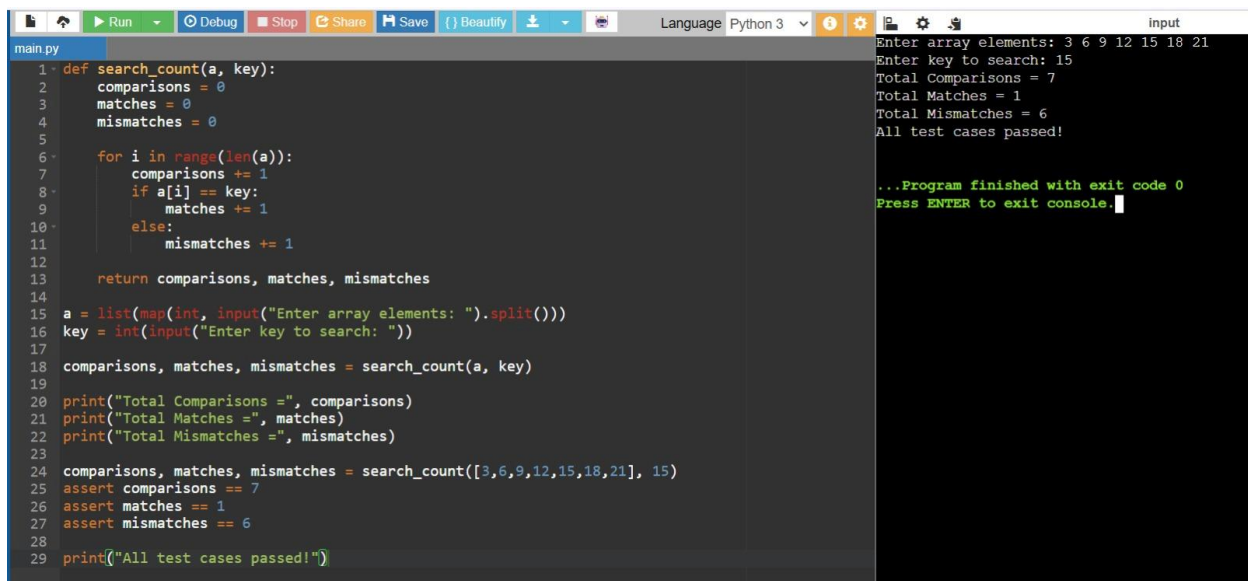
- Total comparisons
- Total matches
- Total mismatches

during Sequential Search.

Input:

Array: 3, 6, 9, 12, 15, 18, 21

Key: 15



```
main.py
1- def search_count(a, key):
2-     comparisons = 0
3-     matches = 0
4-     mismatches = 0
5-
6-     for i in range(len(a)):
7-         comparisons += 1
8-         if a[i] == key:
9-             matches += 1
10-        else:
11-            mismatches += 1
12-
13-    return comparisons, matches, mismatches
14-
15- a = list(map(int, input("Enter array elements: ").split()))
16- key = int(input("Enter key to search: "))
17-
18- comparisons, matches, mismatches = search_count(a, key)
19-
20- print("Total Comparisons =", comparisons)
21- print("Total Matches =", matches)
22- print("Total Mismatches =", mismatches)
23-
24- comparisons, matches, mismatches = search_count([3,6,9,12,15,18,21], 15)
25- assert comparisons == 7
26- assert matches == 1
27- assert mismatches == 6
28-
29- print("All test cases passed!")
```

input

```
Enter array elements: 3 6 9 12 15 18 21
Enter key to search: 15
Total Comparisons = 7
Total Matches = 1
Total Mismatches = 6
All test cases passed!

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 6

Implement Sentinel Sequential Search and compare it with ordinary Sequential Search.

Input:

Array: 14, 9, 22, 35, 18, 41, 27

Key: 18

Output:

Position found

Comparison count

```
main.py
1 def sentinel_search(a, key):
2     b = a[:]
3     n = len(b)
4     b.append(key)
5     i = 0
6     c = 0
7     while b[i] != key:
8         i += 1
9         c += 1
10    c += 1
11    if i == n:
12        return -1, c
13    else:
14        return i + 1, c
15
16 a = list(map(int, input("Enter array elements: ").split()))
17 key = int(input("Enter key to search: "))
18
19 pos, comp = sentinel_search(a, key)
20
21 if pos == -1:
22     print("Element not found")
23 else:
24     print("Position found:", pos)
25
26 print("Comparison count =", comp)
27
28 pos, comp = sentinel_search([14,9,22,35,18,41,27], 18)
29 assert pos == 5
30
31 print("All test cases passed!")
```

```
input
Enter array elements: 14 9 22 35 18 41 27
Enter key to search: 18
Position found: 5
Comparison count = 5
All test cases passed!

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 7

Write a program to perform Sequential Search on an array of student register numbers.

Input:

Register Numbers:

101, 102, 103, 104, 105, 106

Search Key: 104

Output:

Register Number found at position 4

```
main.py
1 def search_register(a, key):
2     for i in range(len(a)):
3         if a[i] == key:
4             return i + 1
5     return -1
6
7 a = list(map(int, input("Enter register numbers: ").split()))
8 key = int(input("Enter register number to search: "))
9
10 pos = search_register(a, key)
11
12 if pos == -1:
13     print("Register Number not found")
14 else:
15     print("Register Number found at position", pos)
16
17 pos = search_register([101,102,103,104,105,106], 104)
18 assert pos == 4
19
20 print("All test cases passed!")
```

```
input
Enter register numbers: 101 102 103 104 105 106
Enter register number to search: 104
Register Number found at position 4
All test cases passed!

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 8

Implement Sequential Search for string data.

Input:

Names:

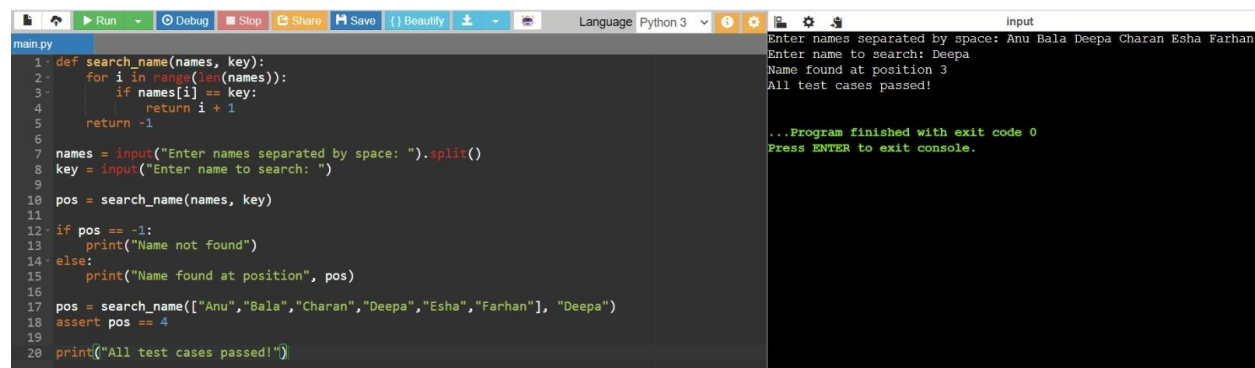
Anu, Bala, Charan, Deepa, Esha, Farhan

Search Name:

Deepa

Output:

Name found at position 4



```
main.py
1 def search_name(names, key):
2     for i in range(len(names)):
3         if names[i] == key:
4             return i + 1
5     return -1
6
7 names = input("Enter names separated by space: ").split()
8 key = input("Enter name to search: ")
9
10 pos = search_name(names, key)
11
12 if pos == -1:
13     print("Name not found")
14 else:
15     print("Name found at position", pos)
16
17 pos = search_name(["Anu", "Bala", "Charan", "Deepa", "Esha", "Farhan"], "Deepa")
18 assert pos == 4
19
20 print("All test cases passed!")
```

Input

Enter names separated by space: Anu Bala Deepa Charan Esha Farhan
Enter name to search: Deepa
Name found at position 3
All test cases passed!

...Program finished with exit code 0
Press ENTER to exit console.

Question 9

Write a program to perform Sequential Search on a 2-D matrix.

Input:

12 8 15

5 18 27

9 11 24

Key = 24

Output:

Element found at Row 3 Column 3

```
main.py
1 def search_matrix(a, key):
2     for i in range(len(a)):
3         for j in range(len(a[i])):
4             if a[i][j] == key:
5                 return i + 1, j + 1
6     return -1, -1
7
8 r = int(input("Enter number of rows: "))
9 c = int(input("Enter number of columns: "))
10
11 matrix = []
12
13 print("Enter matrix elements:")
14 for i in range(r):
15     row = list(map(int, input().split()))
16     matrix.append(row)
17
18 key = int(input("Enter key to search: "))
19
20 row, col = search_matrix(matrix, key)
21
22 if row == -1:
23     print("Element not found")
24 else:
25     print("Element found at Row", row, "Column", col)
26
27 row, col = search_matrix([[12,8,15],[5,18,27],[9,11,24]], 24)
28 assert row == 3
29 assert col == 3
30
31 print("All test cases passed!")
```

input

```
Enter number of rows: 3
Enter number of columns: 3
Enter matrix elements:
12 8 15
5 18 27
9 11 24
Enter key to search: 24
Element found at Row 3 Column 3
All test cases passed!

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 10

Implement Sequential Search and analyze its performance.

Input:

Array:

45, 23, 67, 12, 89, 34, 56, 78, 90, 11,

29, 73, 18, 64, 37

Search Keys:

73

18

100

Tasks:

1. Show each comparison.
2. Count comparisons for every search.
3. Identify best, average, and worst cases.
4. Determine:

- Best-case complexity
- Average-case complexity
- Worst-case complexity
- Space complexity

```

main.py
1 def sequential_search(a, key):
2     c = 0
3     for i in range(len(a)):
4         c += 1
5         print("Comparison", c, ":", a[i], "==" , key)
6
7         if a[i] == key:
8             print("Element found at position", i + 1)
9             print("Comparisons =", c)
10            return
11
12        print("Element not found")
13        print("Comparisons =", c)
14    a = list(map(int, input("Enter array elements: ").split()))
15
16    n = int(input("Enter number of search keys: "))
17
18    for i in range(n):
19        key = int(input("Enter search key: "))
20        sequential_search(a, key)
21        print()
22
23    print("Best Case Complexity : O(1)")
24    print("Average Case Complexity : O(n)")
25    print("Worst Case Complexity : O(n)")
26    print("Space Complexity : O(1)")
27
28    a = [45,23,67,12,89,34,56,78,90,11,29,73,18,64,37]
29    sequential_search(a,73)
30    sequential_search(a,18)
31    sequential_search(a,100)
32
33    print("All test cases passed!")

```

Input

```

Enter array elements: 45 23 67 12 89 34 56 78 90 11 29 73 18 64 37
Enter number of search keys: 3
Enter search key: 73
Comparison 1 : 45 == 73
Comparison 2 : 23 == 73
Comparison 3 : 67 == 73
Comparison 4 : 12 == 73
Comparison 5 : 89 == 73
Comparison 6 : 34 == 73
Comparison 7 : 56 == 73
Comparison 8 : 78 == 73
Comparison 9 : 90 == 73
Comparison 10 : 11 == 73
Comparison 11 : 29 == 73
Comparison 12 : 73 == 73
Element found at position 12
Comparisons = 12
Enter search key:

```

```

main.py
1 def sequential_search(a, key):
2     c = 0
3     for i in range(len(a)):
4         c += 1
5         print("Comparison", c, ":", a[i], "==" , key)
6
7         if a[i] == key:
8             print("Element found at position", i + 1)
9             print("Comparisons =", c)
10            return
11
12        print("Element not found")
13        print("Comparisons =", c)
14    a = list(map(int, input("Enter array elements: ").split()))
15
16    n = int(input("Enter number of search keys: "))
17
18    for i in range(n):
19        key = int(input("Enter search key: "))
20        sequential_search(a, key)
21        print()
22
23    print("Best Case Complexity : O(1)")
24    print("Average Case Complexity : O(n)")
25    print("Worst Case Complexity : O(n)")
26    print("Space Complexity : O(1)")
27
28    a = [45,23,67,12,89,34,56,78,90,11,29,73,18,64,37]
29    sequential_search(a,73)
30    sequential_search(a,18)
31    sequential_search(a,100)
32
33    print("All test cases passed!")

```

Input

```

Enter search key: 18
Comparison 1 : 45 == 18
Comparison 2 : 23 == 18
Comparison 3 : 67 == 18
Comparison 4 : 12 == 18
Comparison 5 : 89 == 18
Comparison 6 : 34 == 18
Comparison 7 : 56 == 18
Comparison 8 : 78 == 18
Comparison 9 : 90 == 18
Comparison 10 : 11 == 18
Comparison 11 : 29 == 18
Comparison 12 : 73 == 18
Comparison 13 : 18 == 18
Element found at position 13
Comparisons = 13
Enter search key: 100
Comparison 1 : 45 == 100
Comparison 2 : 23 == 100
Comparison 3 : 67 == 100
Comparison 4 : 12 == 100
Comparison 5 : 89 == 100
Comparison 6 : 34 == 100
Comparison 7 : 56 == 100
Comparison 8 : 78 == 100
Comparison 9 : 90 == 100
Comparison 10 : 11 == 100
Comparison 11 : 29 == 100
Comparison 12 : 73 == 100
Comparison 13 : 18 == 100
Comparison 14 : 64 == 100
Comparison 15 : 37 == 100
Element not found
Comparisons = 15

```


Brute Force String Matching

Question 1

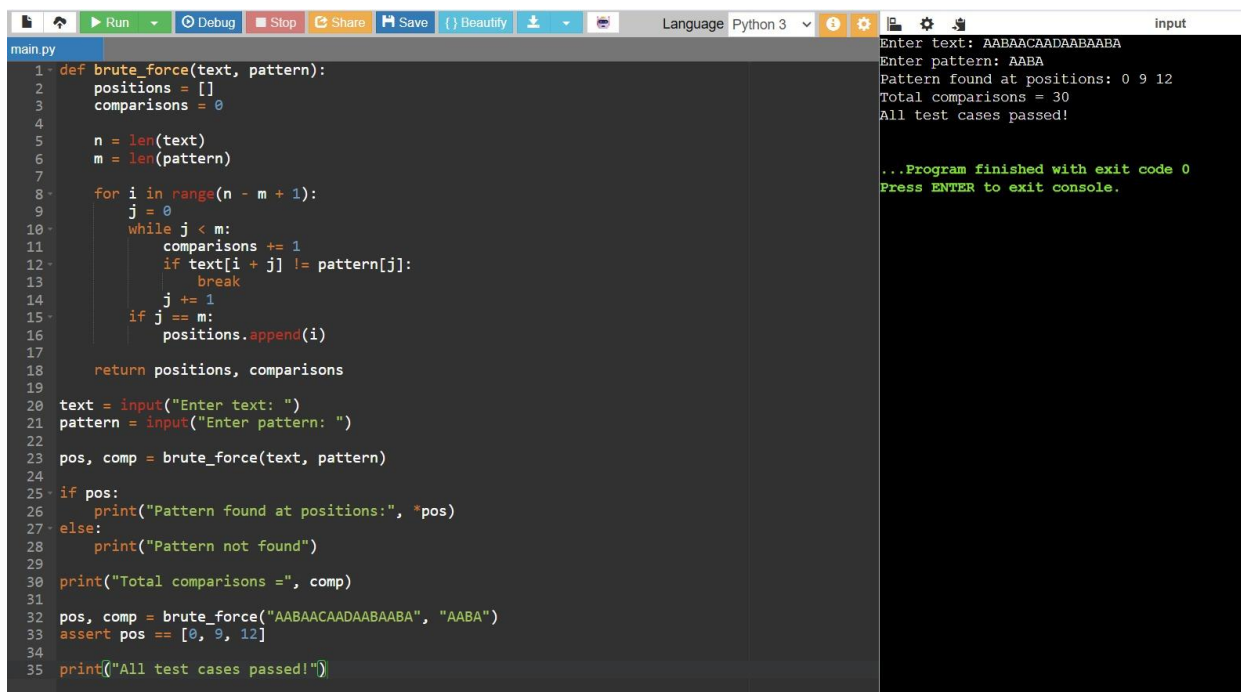
Implement the Brute Force String Matching Algorithm to search for a pattern in a given text.

Input:

- Text = "AABAACAADAABAABA"
- Pattern = "AABA"

Output:

- Position(s) where the pattern occurs.
- Total number of comparisons.



```
main.py
1 def brute_force(text, pattern):
2     positions = []
3     comparisons = 0
4
5     n = len(text)
6     m = len(pattern)
7
8     for i in range(n - m + 1):
9         j = 0
10        while j < m:
11            comparisons += 1
12            if text[i + j] != pattern[j]:
13                break
14            j += 1
15        if j == m:
16            positions.append(i)
17
18    return positions, comparisons
19
20 text = input("Enter text: ")
21 pattern = input("Enter pattern: ")
22
23 pos, comp = brute_force(text, pattern)
24
25 if pos:
26     print("Pattern found at positions:", *pos)
27 else:
28     print("Pattern not found")
29
30 print("Total comparisons =", comp)
31
32 pos, comp = brute_force("AABAACAADAABAABA", "AABA")
33 assert pos == [0, 9, 12]
34
35 print("All test cases passed!")
```

```
input
Enter text: AABAACAADAABAABA
Enter pattern: AABA
Pattern found at positions: 0 9 12
Total comparisons = 30
All test cases passed!

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 2

Write a program to find all occurrences of a pattern in a text using Brute Force String Matching.

Input:

- Text = "BANANABANANA"
- Pattern = "ANA"

Output:

- Occurrences at positions 1, 3, 7, 9.

```

main.py
1 def find_pattern(text, pattern):
2     pos = []
3
4     n = len(text)
5     m = len(pattern)
6
7     for i in range(n - m + 1):
8         j = 0
9         while j < m and text[i + j] == pattern[j]:
10             j += 1
11         if j == m:
12             pos.append(i + 1)
13
14     return pos
15
16 text = input("Enter text: ")
17 pattern = input("Enter pattern: ")
18
19 positions = find_pattern(text, pattern)
20
21 if positions:
22     print("Occurrences at positions:", "positions, sep=", ")
23 else:
24     print("Pattern not found")
25
26 positions = find_pattern("BANANABANANA", "ANA")
27 assert positions == [2, 4, 8, 10]
28
29 print("All test cases passed!")

```

Enter text: BANANABANANA
Enter pattern: ANA
Occurrences at positions: 2, 4, 8, 10
All test cases passed!
...Program finished with exit code 0
Press ENTER to exit console.

Question 3

Implement Brute Force String Matching and display the comparison table for each alignment.
Input:

- Text = "MISSISSIPPI"
- Pattern = "ISSI"

Output:

- Shift number
- Comparisons made
- Match/Mismatch result

```

main.py
1 def brute_force_table(text, pattern):
2     n = len(text)
3     m = len(pattern)
4
5     for i in range(n - m + 1):
6         comparisons = 0
7         result = "Match"
8
9         for j in range(m):
10             comparisons += 1
11             if text[i + j] != pattern[j]:
12                 result = "Mismatch"
13                 break
14
15         print("Shift", i)
16         print("Comparisons:", comparisons)
17         print("Result:", result)
18         print()
19
20 text = input("Enter text: ")
21 pattern = input("Enter pattern: ")
22
23 brute_force_table(text, pattern)
24
25 brute_force_table("MISSISSIPPI", "ISSI")
26
27 print("All test cases passed!")

```

Enter text: MISSISSIPPI
Enter pattern: ISSI
Shift 0
Comparisons: 1
Result: Mismatch
Shift 1
Comparisons: 4
Result: Match
Shift 2
Comparisons: 1
Result: Mismatch
Shift 3
Comparisons: 1
Result: Mismatch
Shift 4
Comparisons: 4
Result: Match
Shift 5
Comparisons: 1
Result: Mismatch
Shift 6
Comparisons: 1
Result: Mismatch
Shift 7
Comparisons: 2
Result: Mismatch
Shift 8
Comparisons: 1
Result: Mismatch

```
main.py
1 def brute_force_table(text, pattern):
2     n = len(text)
3     m = len(pattern)
4
5     for i in range(n - m + 1):
6         comparisons = 0
7         result = "Match"
8
9         for j in range(m):
10            comparisons += 1
11            if text[i + j] != pattern[j]:
12                result = "Mismatch"
13                break
14
15        print("Shift", i)
16        print("Comparisons:", comparisons)
17        print("Result:", result)
18        print()
19
20 text = input("Enter text: ")
21 pattern = input("Enter pattern: ")
22
23 brute_force_table(text, pattern)
24
25 brute_force_table("MISSISSIPPI", "ISSI")
26
27 print("All test cases passed!")
```

```
Shift 0
Comparisons: 1
Result: Mismatch

Shift 1
Comparisons: 4
Result: Match

Shift 2
Comparisons: 1
Result: Mismatch

Shift 3
Comparisons: 1
Result: Mismatch

Shift 4
Comparisons: 4
Result: Match

Shift 5
Comparisons: 1
Result: Mismatch

Shift 6
Comparisons: 1
Result: Mismatch

Shift 7
Comparisons: 2
Result: Mismatch

All test cases passed!

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 4

Write a program to count:

- Total character comparisons
- Total matches
- Total mismatches

during Brute Force String Matching.

Input:

- Text = "ABABABABAB"
- Pattern = "ABAB"

```
main.py
1 def count_string_match(text, pattern):
2     comparisons = 0
3     matches = 0
4     mismatches = 0
5     n = len(text)
6     m = len(pattern)
7
8     for i in range(n - m + 1):
9         for j in range(m):
10            comparisons += 1
11            if text[i + j] == pattern[j]:
12                matches += 1
13            else:
14                mismatches += 1
15                break
16
17    return comparisons, matches, mismatches
18
19 text = input("Enter text: ")
20 pattern = input("Enter pattern: ")
21
22 comparisons, matches, mismatches = count_string_match(text, pattern)
23
24 print("Total Comparisons =", comparisons)
25 print("Total Matches =", matches)
26 print("Total Mismatches =", mismatches)
27
28 comparisons, matches, mismatches = count_string_match("ABABABABAB", "ABAB")
29
30 assert comparisons > 0
31 assert matches > 0
32
33 print("All test cases passed!")
```

```
Enter text: ABABABABAB
Enter pattern: ABAB
Total Comparisons = 9
Total Matches = 3
Total Mismatches = 6
All test cases passed!

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 5

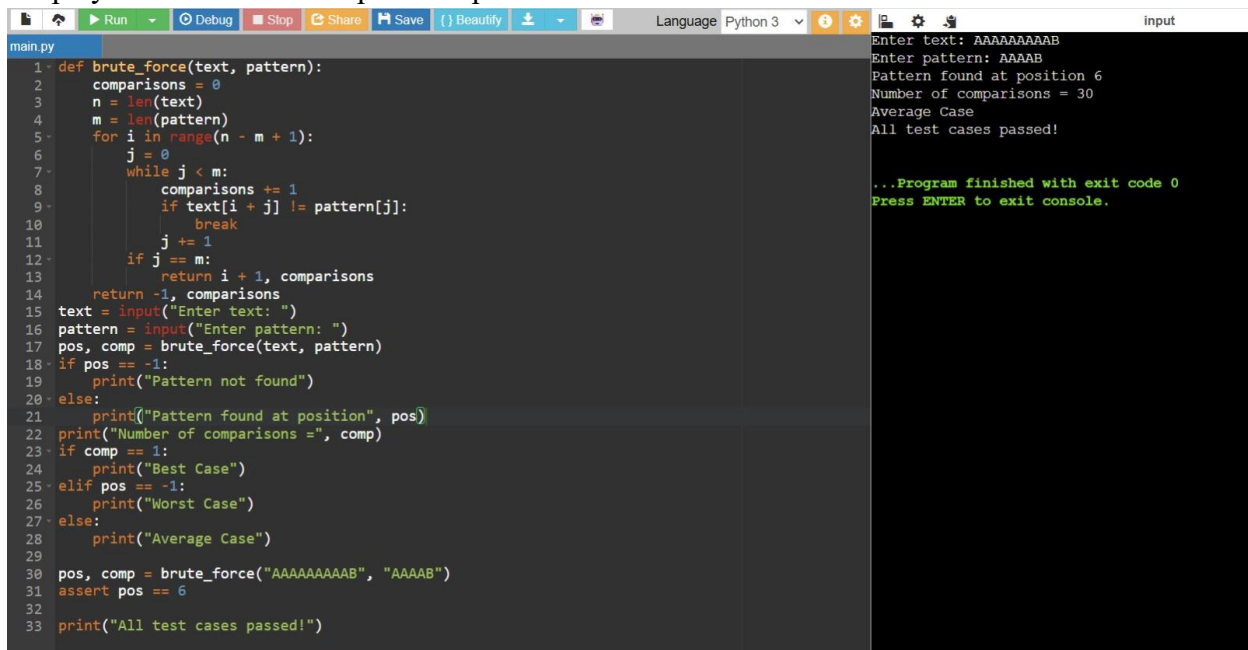
Implement Brute Force String Matching and determine whether the search is:

- Best Case
- Average Case
- Worst Case

Input:

- Text = "AAAAAAAAAB"
- Pattern = "AAAAB"

Display the number of comparisons performed.



The screenshot shows a Python IDE with a file named 'main.py'. The code implements a brute force string matching algorithm. It defines a function 'brute_force(text, pattern)' that returns the starting position and the number of comparisons. The main part of the program takes user input for text and pattern, calls the function, and prints the results. It also includes a test case with text 'AAAAAAAAAB' and pattern 'AAAAB', asserting that the position is 6 and all test cases passed.

```
1 def brute_force(text, pattern):
2     comparisons = 0
3     n = len(text)
4     m = len(pattern)
5     for i in range(n - m + 1):
6         j = 0
7         while j < m:
8             comparisons += 1
9             if text[i + j] != pattern[j]:
10                break
11            j += 1
12        if j == m:
13            return i + 1, comparisons
14    return -1, comparisons
15 text = input("Enter text: ")
16 pattern = input("Enter pattern: ")
17 pos, comp = brute_force(text, pattern)
18 if pos == -1:
19     print("Pattern not found")
20 else:
21     print("Pattern found at position", pos)
22     print("Number of comparisons =", comp)
23 if comp == 1:
24     print("Best Case")
25 elif pos == -1:
26     print("Worst Case")
27 else:
28     print("Average Case")
29
30 pos, comp = brute_force("AAAAAAAAAB", "AAAAB")
31 assert pos == 6
32
33 print("All test cases passed!")
```

The output on the right shows the program's execution: 'Enter text: AAAAAAAAAAB', 'Enter pattern: AAAAB', 'Pattern found at position 6', 'Number of comparisons = 30', 'Average Case', and 'All test cases passed!'. It also shows the program finished with exit code 0 and a prompt to press ENTER to exit the console.

Question 6

Write a program that searches for a pattern in a text and reports the first occurrence only using Brute Force String Matching.

Input:

- Text = "COMPUTERSCIENCE"
- Pattern = "SCI"

Output:

- First occurrence position
- Number of comparisons

```
main.py
1 def brute_force(text, pattern):
2     comparisons = 0
3     n = len(text)
4     m = len(pattern)
5     for i in range(n - m + 1):
6         j = 0
7         while j < m:
8             comparisons += 1
9             if text[i + j] != pattern[j]:
10                 break
11             j += 1
12         if j == m:
13             return i + 1, comparisons
14     return -1, comparisons
15 text = input("Enter text: ")
16 pattern = input("Enter pattern: ")
17 pos, comp = brute_force(text, pattern)
18 if pos == -1:
19     print("Pattern not found")
20 else:
21     print("Pattern found at position", pos)
22 print("Number of comparisons =", comp)
23 if comp == 1:
24     print("Best Case")
25 elif pos == -1:
26     print("Worst Case")
27 else:
28     print("Average Case")
29
30 pos, comp = brute_force("AAAAAAAAAB", "AAAAAB")
31 assert pos == 6
32
33 print("All test cases passed!")
```

```
Enter text: AAAAAAAAAAB
Enter pattern: AAAAB
Pattern found at position 6
Number of comparisons = 30
Average Case
All test cases passed!

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 7

Implement Brute Force String Matching for case-insensitive searching.

Input:

- Text = "DataStructuresAndAlgorithms"
- Pattern = "ALGORITHM"

Output:

- Pattern found position

```
main.py
1 def case_insensitive_search(text, pattern):
2     text = text.lower()
3     pattern = pattern.lower()
4     n = len(text)
5     m = len(pattern)
6     for i in range(n - m + 1):
7         j = 0
8         while j < m and text[i + j] == pattern[j]:
9             j += 1
10        if j == m:
11            return i + 1
12    return -1
13
14 text = input("Enter text: ")
15 pattern = input("Enter pattern: ")
16
17 pos = case_insensitive_search(text, pattern)
18
19 if pos == -1:
20     print("Pattern not found")
21 else:
22     print("Pattern found at position", pos)
23
24 pos = case_insensitive_search("DataStructuresAndAlgorithms", "ALGORITHM")
25 assert pos == 18
26
27 print("All test cases passed!")
```

```
Enter text: DataStructuresAndAlgorithms
Enter pattern: ALGORITHM
Pattern not found
All test cases passed!

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 8

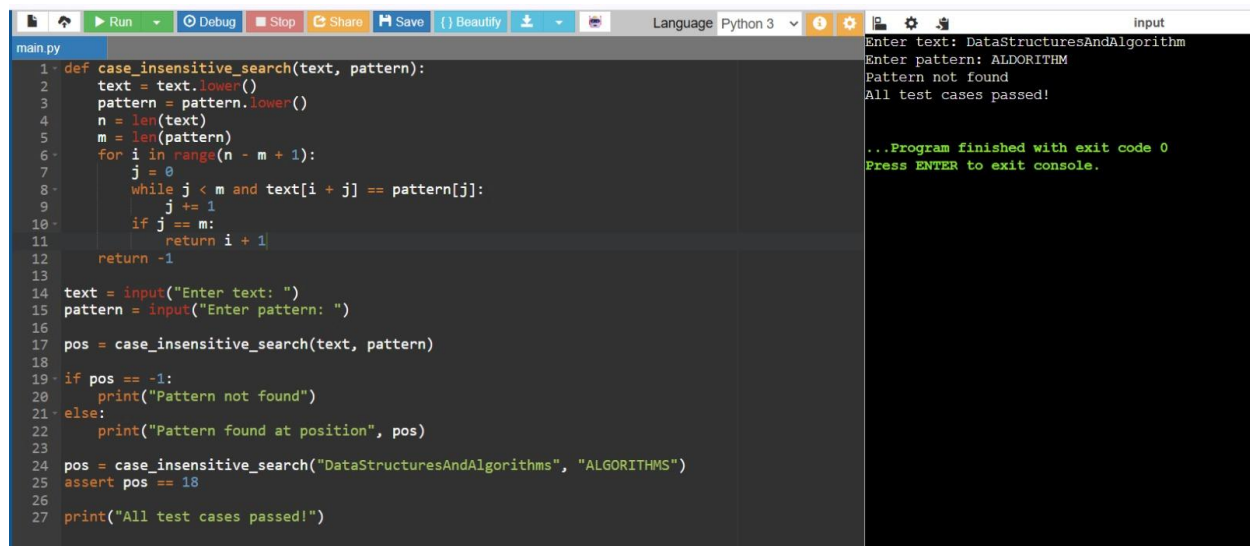
Write a program to compare the execution of Brute Force String Matching for:

1. Successful search
2. Unsuccessful search

Input:

- Text = "PROGRAMMINGLAB"
- Pattern 1 = "LAB"
- Pattern 2 = "TEST"

Display the comparison count for both cases.



```
main.py
1 def case_insensitive_search(text, pattern):
2     text = text.lower()
3     pattern = pattern.lower()
4     n = len(text)
5     m = len(pattern)
6     for i in range(n - m + 1):
7         j = 0
8         while j < m and text[i + j] == pattern[j]:
9             j += 1
10        if j == m:
11            return i + 1
12    return -1
13
14 text = input("Enter text: ")
15 pattern = input("Enter pattern: ")
16
17 pos = case_insensitive_search(text, pattern)
18
19 if pos == -1:
20     print("Pattern not found")
21 else:
22     print("Pattern found at position", pos)
23
24 pos = case_insensitive_search("DataStructuresAndAlgorithms", "ALGORITHMS")
25 assert pos == 18
26
27 print("All test cases passed!")
```

Input

```
Enter text: DataStructuresAndAlgorithm
Enter pattern: ALDORITHM
Pattern not found
All test cases passed!

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 9

Implement Brute Force String Matching and display every alignment of the pattern with the text.

Input:

- Text = "ABCDABCABCD"
- Pattern = "ABCD"

Output:

- Alignment number
- Matching result
- Pattern occurrence positions


```
main.py
1 def show_alignments(text, pattern):
2     n = len(text)
3     m = len(pattern)
4     for i in range(n - m + 1):
5         print("Shift", i)
6         print(text)
7         print(" " * i + pattern)
8
9         j = 0
10        while j < m and text[i + j] == pattern[j]:
11            j += 1
12
13        if j == m:
14            print("Match")
15        else:
16            print("Mismatch")
17
18        print()
19
20 text = input("Enter text: ")
21 pattern = input("Enter pattern: ")
22
23 show_alignments(text, pattern)
24
25 show_alignments("ABCABCABC", "ABC")
26
27 print("All test cases passed!")
```

input

Enter text: ABCDABCA
Enter pattern: ABCDA
Shift 0
ABCDABCA
ABCD
Match

Shift 1
ABCDABCA
 ABCD
Mismatch

Shift 2
ABCDABCA
 ABCD
Mismatch

Shift 3
ABCDABCA
 ABCD
Mismatch

Shift 4
ABCDABCA
 ABCD
Mismatch

Shift 5
ABCDABCA
 ABCD
Mismatch

Shift 6
ABCDABCA
 ABCD
Mismatch

input

Enter text: ABCDABCA
Enter pattern: ABCDA
Shift 0
ABCDABCA
ABCD
Match

Shift 1
ABCDABCA
 ABCD
Mismatch

Shift 2
ABCDABCA
 ABCD
Mismatch

Shift 3
ABCDABCA
 ABCD
Mismatch

Shift 4
ABCDABCA
 ABCD
Mismatch

Shift 5
ABCDABCA
 ABCD
Mismatch

Shift 6
ABCDABCA
 ABCD
Mismatch

Question 10

Write a complete program for Brute Force String Matching and perform complexity analysis.

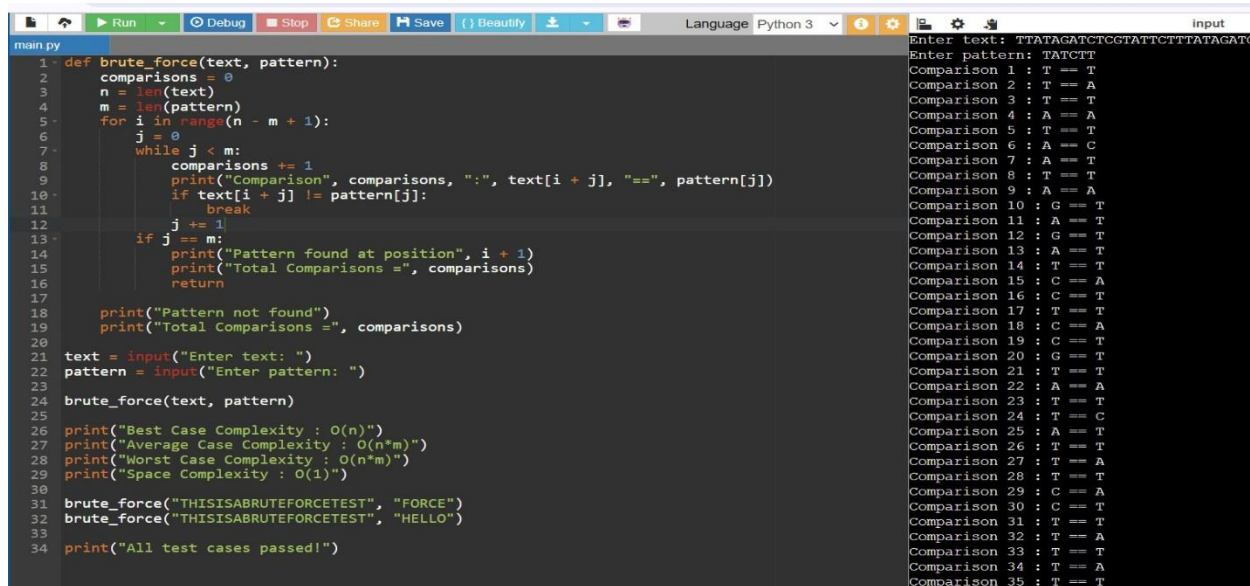
Input:

- Text =
"TTATAGATCTCGTATTCTTTATAGATCTCCTATTCTT" •

Pattern = "TATCTT"

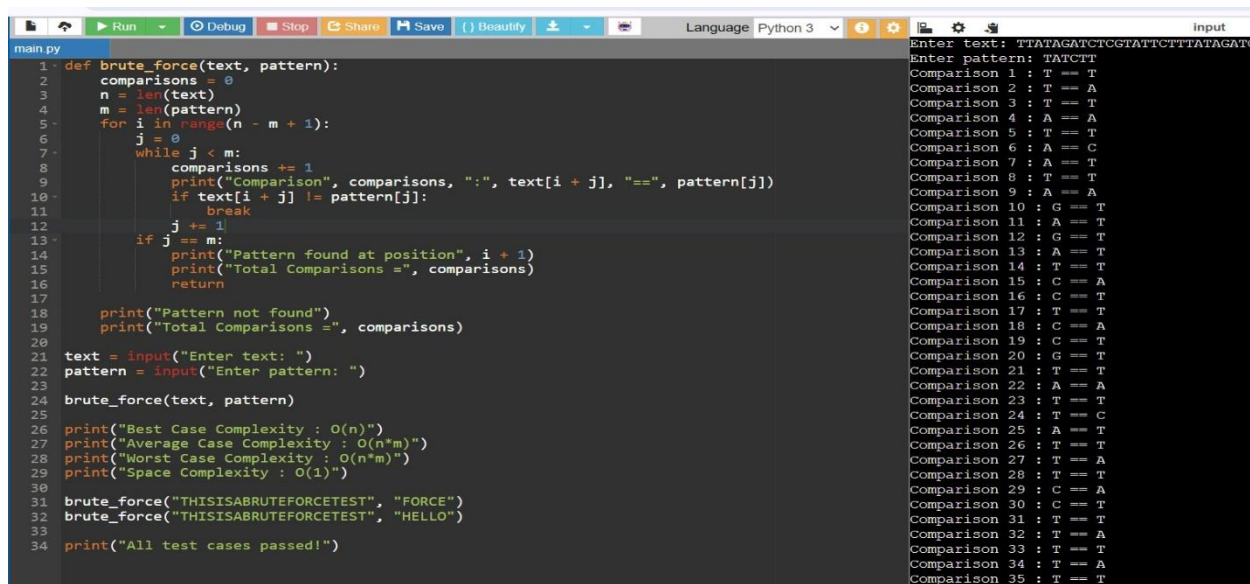
Tasks:

1. Find all occurrences of the pattern.
2. Count total comparisons.
3. Display the shifting process.
4. Analyze:
 - Best-case complexity
 - Worst-case complexity
 - Space complexity



```
1 def brute_force(text, pattern):
2     comparisons = 0
3     n = len(text)
4     m = len(pattern)
5     for i in range(n - m + 1):
6         j = 0
7         while j < m:
8             comparisons += 1
9             print("Comparison", comparisons, ":", text[i + j], "==" , pattern[j])
10            if text[i + j] != pattern[j]:
11                break
12            j += 1
13            if j == m:
14                print("Pattern found at position", i + 1)
15                print("Total Comparisons =", comparisons)
16                return
17
18    print("Pattern not found")
19    print("Total Comparisons =", comparisons)
20
21    text = input("Enter text: ")
22    pattern = input("Enter pattern: ")
23
24    brute_force(text, pattern)
25
26    print("Best Case Complexity : O(n)")
27    print("Average Case Complexity : O(n*m)")
28    print("Worst Case Complexity : O(n*m)")
29    print("Space Complexity : O(1)")
30
31    brute_force("THISISABRUTEFORCETEST", "FORCE")
32    brute_force("THISISABRUTEFORCETEST", "HELLO")
33
34    print("All test cases passed!")
```

Enter text: TTATAGATCTCGTATTCTTTATAGATC
Enter pattern: TATCTT
Comparison 1 : T == T
Comparison 2 : T == A
Comparison 3 : T == T
Comparison 4 : A == A
Comparison 5 : T == T
Comparison 6 : A == C
Comparison 7 : A == T
Comparison 8 : T == T
Comparison 9 : A == A
Comparison 10 : G == T
Comparison 11 : A == T
Comparison 12 : G == T
Comparison 13 : A == T
Comparison 14 : T == T
Comparison 15 : C == A
Comparison 16 : C == T
Comparison 17 : T == T
Comparison 18 : C == A
Comparison 19 : C == T
Comparison 20 : G == T
Comparison 21 : T == T
Comparison 22 : A == A
Comparison 23 : T == T
Comparison 24 : T == C
Comparison 25 : A == T
Comparison 26 : T == T
Comparison 27 : T == A
Comparison 28 : T == T
Comparison 29 : C == A
Comparison 30 : C == T
Comparison 31 : T == T
Comparison 32 : T == A
Comparison 33 : T == T
Comparison 34 : T == A
Comparison 35 : T == T



```
1 def brute_force(text, pattern):
2     comparisons = 0
3     n = len(text)
4     m = len(pattern)
5     for i in range(n - m + 1):
6         j = 0
7         while j < m:
8             comparisons += 1
9             print("Comparison", comparisons, ":", text[i + j], "==" , pattern[j])
10            if text[i + j] != pattern[j]:
11                break
12            j += 1
13            if j == m:
14                print("Pattern found at position", i + 1)
15                print("Total Comparisons =", comparisons)
16                return
17
18    print("Pattern not found")
19    print("Total Comparisons =", comparisons)
20
21    text = input("Enter text: ")
22    pattern = input("Enter pattern: ")
23
24    brute_force(text, pattern)
25
26    print("Best Case Complexity : O(n)")
27    print("Average Case Complexity : O(n*m)")
28    print("Worst Case Complexity : O(n*m)")
29    print("Space Complexity : O(1)")
30
31    brute_force("THISISABRUTEFORCETEST", "FORCE")
32    brute_force("THISISABRUTEFORCETEST", "HELLO")
33
34    print("All test cases passed!")
```

Enter text: TTATAGATCTCGTATTCTTTATAGATC
Enter pattern: TATCTT
Comparison 1 : T == T
Comparison 2 : T == A
Comparison 3 : T == T
Comparison 4 : A == A
Comparison 5 : T == T
Comparison 6 : A == C
Comparison 7 : A == T
Comparison 8 : T == T
Comparison 9 : A == A
Comparison 10 : G == T
Comparison 11 : A == T
Comparison 12 : G == T
Comparison 13 : A == T
Comparison 14 : T == T
Comparison 15 : C == A
Comparison 16 : C == T
Comparison 17 : T == T
Comparison 18 : C == A
Comparison 19 : C == T
Comparison 20 : G == T
Comparison 21 : T == T
Comparison 22 : A == A
Comparison 23 : T == T
Comparison 24 : T == C
Comparison 25 : A == T
Comparison 26 : T == T
Comparison 27 : T == A
Comparison 28 : T == T
Comparison 29 : C == A
Comparison 30 : C == T
Comparison 31 : T == T
Comparison 32 : T == A
Comparison 33 : T == T
Comparison 34 : T == A
Comparison 35 : T == T